

PROFESSIONAL SUMMARY

Backend-focused Software Engineering student at MUET with 4 production-grade Java projects demonstrating deep expertise in Spring Boot, REST API design, Spring Security, Spring Data JPA, and MySQL/PostgreSQL. Proven ability to architect layered systems with strict separation of concerns, enforce SQL injection prevention via PreparedStatements, and deliver full CRUD lifecycles with proper HTTP semantics. Actively seeking a Software Engineering role to contribute backend development skills to a product team.

TECHNICAL SKILLS

Languages: Java (Advanced) · C++ (Intermediate) · SQL (Advanced) · HTML5 · CSS3 · JavaScript
Frameworks: Spring Boot · Spring MVC · Spring Data JPA · Spring Security · Spring Actuator · Servlets · JSP
Databases: MySQL · PostgreSQL · JDBC (PreparedStatements, Transaction Mgmt) · H2 Embedded Database
Tools: Git/GitHub · Maven · Postman · Docker · Docker Compose · Swagger · Apache Tomcat · IntelliJ IDEA · VS Code
Concepts: OOP · MVC Architecture · DAO/DTO Patterns · REST API Design · JUnit · Agile · Software Testing · API Testing

EDUCATION

B.E. in Software Engineering Aug 2023 – Aug 2027 (Expected)
Mehran University of Engineering & Technology (MUET), Jamshoro, Sindh
Coursework: OOP, Data Structures, Database Systems, Software Engineering, Web Technologies, Discrete Mathematics

Intermediate in Pre-Engineering 2019 – 2021
AIMS School and College, Matli, Sindh

PROJECTS

LetsReadBlogs — Full-Stack Blog Platform | Java 25 · Spring Boot 4 · Spring Security · JWT · Feb 2026 – May 2026
MapStruct · PostgreSQL · React · TypeScript · Docker
github.com/Tahir-Dars/letsReadBlogs

- Architected a full-stack blog platform (148 commits) with a 7-package Spring Boot backend — **config, controller, domain, mapper, repository, security, service** — enforcing strict separation of concerns across every layer.
- Implemented **JWT-based stateless authentication** using JJWT 0.13.0 with Spring Security, securing all write endpoints (create/edit/delete posts, categories, tags) while keeping public read routes open.
- Replaced boilerplate DTO↔Entity conversions with **MapStruct 1.6.3** compile-time mappers, decoupling API contracts from internal domain models — eliminating runtime reflection overhead entirely.
- Containerised the full data layer with **Docker Compose**: PostgreSQL on port 5432 + Adminer admin panel on port 8888, enabling zero-config local setup and reproducible environments.
- Delivered **15+ REST endpoints** under `/api/v1/*` covering full CRUD for posts, categories, and tags; enforced Spring Validation on all incoming payloads and wired Spring Actuator health hooks for observability.
- Integrated a **React + TypeScript + Vite** frontend (deployed on Vercel) with TipTap rich-text editor, Axios API client, and DOMPurify sanitisation — live at letsreadblogshere.vercel.app.

Banking Customer REST API | Java · Spring Boot · Spring Data JPA · Spring Security · H2 · Maven Jan 2026
github.com/Tahir-Dars/BankCustomerAPI

- Architected RESTful API for full banking customer lifecycle (CRUD) using a 4-tier Spring Boot structure: Controller → Service → Repository → Model with clean layer boundaries.
- Integrated Spring Security authentication and Spring Actuator health monitoring, establishing endpoint protection and production observability from project inception.
- Delivered complete API with proper HTTP semantics (200, 201, 400, 404, 500) and consistent JSON response contracts across all endpoints.

Full-Stack To-Do List Web Application | Java · Servlets · JSP · JDBC · MySQL · Maven · Tomcat · Nov 2025 – Dec 2025
Bootstrap 5
github.com/Tahir-Dars/todolistUsingServlet

- Shipped end-to-end task management app: designed MySQL schema, built 3 HTTP Servlet endpoints (CRUD), deployed WAR to Apache Tomcat, and integrated Bootstrap 5 responsive UI.
- Designed 4-layer architecture (Controller → DAO → DB Connector → Entity Model) achieving clean separation of HTTP handling from data access logic with zero inter-layer coupling.

PreparedStatement

github.com/Tahir-Dars/BankingSystem_JDBC

- Engineered production-grade console banking app with account creation, deposit/withdrawal, and fund transfer — implementing explicit JDBC transaction control with manual commit/rollback on all balance mutations.
- Achieved **ZERO data corruption across 50+ test scenarios** by enforcing atomic transaction boundaries; validated that partial-transfer failures roll back correctly under all tested conditions.
- Built secure authentication using PreparedStatement across all database operations, eliminating SQL injection vectors entirely in all 3 transaction types.

KEY ACHIEVEMENTS

- **Full-Stack Expansion:** Extended from pure backend (Spring Boot REST APIs) to full-stack delivery with React + TypeScript frontend, Docker Compose infra, and Vercel deployment within a single project.
- **Rapid Technical Progression:** Advanced from raw JDBC/console apps to Spring Boot REST APIs with Security, JWT, and MapStruct integration within 12 months of self-directed learning.
- **Production-Quality Portfolio:** Delivered 4 complete, publicly documented GitHub projects meeting professional architecture standards — layered design, security, and test coverage in every project.
- **Data Integrity:** Achieved ZERO data corruption across 50+ banking system test scenarios via explicit JDBC transaction boundaries and atomic operation validation.
- **Security-First Development:** Applied SQL injection prevention (PreparedStatement) across 100% of database operations and JWT-based stateless auth in all secured projects.

LANGUAGES

English — Professional Proficiency | Urdu — Native | Sindhi — Native